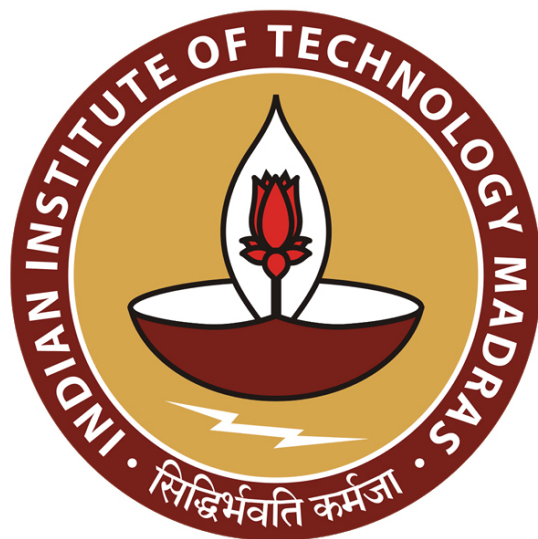


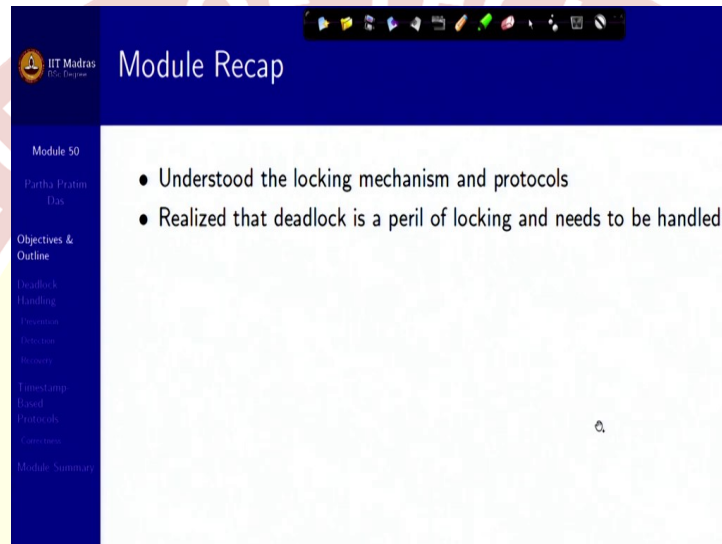


IIT Madras
ONLINE DEGREE

Database Management System
Professor Partha Pratim Das
Department of Computer Science and Engineering
Indian Institute of Technology Kharagpur
Concurrency Control 2

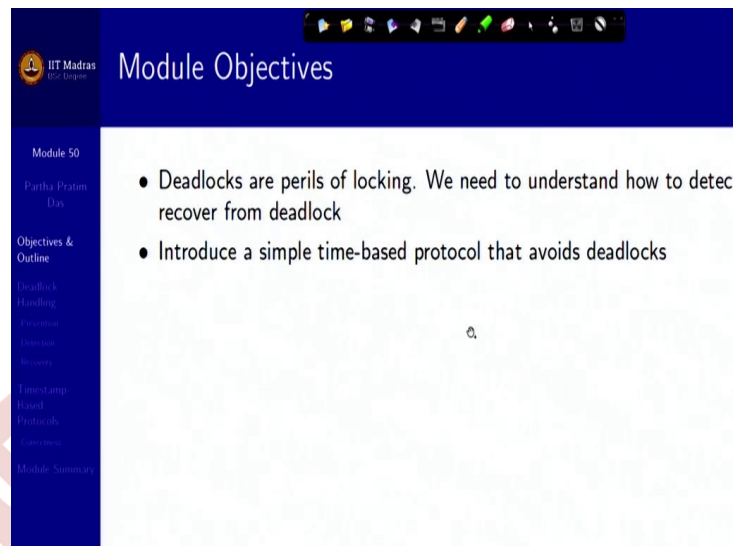
Welcome to module 50 of database management systems course in the IIT Madras online B.Sc. program.

(Refer Slide Time: 00:24)



We have been discussing about concurrency control, and in the last module we understand, we took an understanding of the locking mechanism and protocols, particularly the two phase locking protocol, and realize that while the locking guarantees isolation, locking does improve on the serializability, locking ensures that we can have consistency all the time, but it has a big problem of deadlock. And that can happen very often very easily even in small examples and so on.

(Refer Slide Time: 01:04)



IIT Madras
For Engineers

Module Objectives

Module 50
Partha Pratim Das

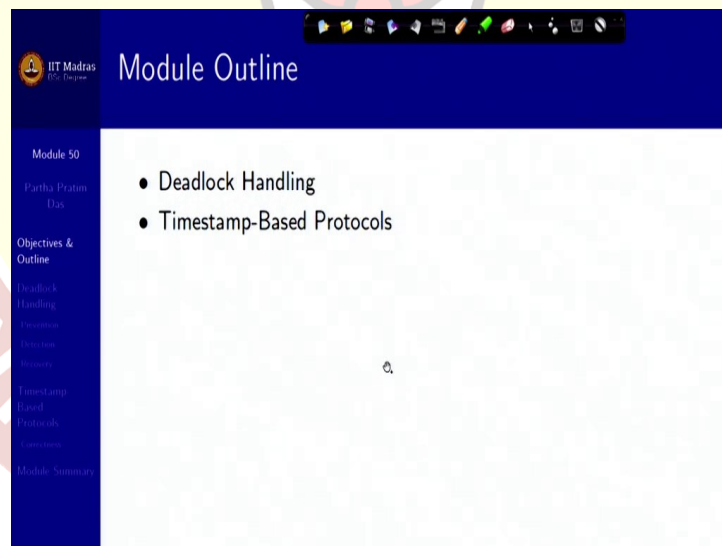
Objectives & Outline

- Deadlocks
- Handling
- Prevention
- Detection
- Recovery
- Timestamp-Based
- Protocols
- Consistency
- Module Summary

- Deadlocks are perils of locking. We need to understand how to detect and recover from deadlock
- Introduce a simple time-based protocol that avoids deadlocks

So then, the question is how do we detect deadlock, or even better still, can we take strategies to prevent deadlock, or recover from deadlock, and because we will have to live with deadlock, there is no other option.

(Refer Slide Time: 01:28)



IIT Madras
For Engineers

Module Outline

Module 50
Partha Pratim Das

Objectives & Outline

- Deadlocks
- Handling
- Prevention
- Detection
- Recovery
- Timestamp-Based
- Protocols
- Consistency
- Module Summary

- Deadlock Handling
- Timestamp-Based Protocols

So, also there could be some simple protocols which can avoid deadlock itself. So it is primarily about deadlock handling.

(Refer Slide Time: 01:32)

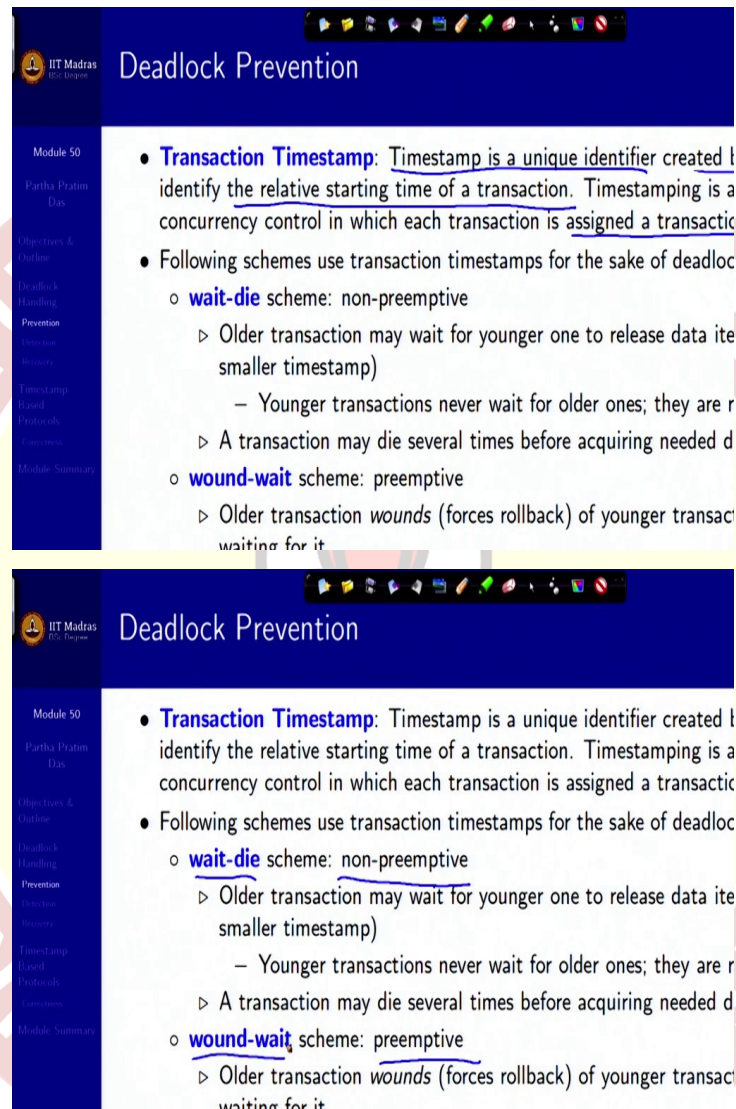


So deadlock system is deadlocked, again, just to remind you, if there is a set of transactions that each, every transaction in the set is waiting for another transaction in the set. Simplest can happen between two transactions but it can happen between multiple transactions. So deadlock prevention is a protocol that the system will never enter into a deadlock state.

So can we, can we guarantee that? Can we do something to prevent the onset of the deadlock? So this could be require each transaction locks all data items before it begin transactions. Usually difficult, and particularly if a transaction is dealing with large number of data items, and may significantly reduce the concurrency in the system.

We could also impose partial ordering of data items that, and required the transaction can lock data items only in the specified partial order. So these are some of the strategies, we will not go much into them.

(Refer Slide Time: 02:54)



Deadlock Prevention

- **Transaction Timestamp:** Timestamp is a unique identifier created to identify the relative starting time of a transaction. Timestamping is a concurrency control in which each transaction is assigned a transaction timestamp.
- Following schemes use transaction timestamps for the sake of deadlock prevention:
 - **wait-die** scheme: non-preemptive
 - ▷ Older transaction may wait for younger one to release data item (smaller timestamp)
 - Younger transactions never wait for older ones; they are rolled back.
 - ▷ A transaction may die several times before acquiring needed data item.
 - **wound-wait** scheme: preemptive
 - ▷ Older transaction *wounds* (forces rollback) of younger transaction waiting for it.

Now, a pair of strategy for deadlock prevention which is very effective, is based on time-stamps. So what is the time-stamp? A time-stamp is a unique identifier which is created by the database to identify the relative starting time of a transaction. It may not exactly be the clock time, but it just says which transaction was earlier, which transaction was later depending on when the transaction started executing.

So it is a method of concurrency control, which can use this transaction time-stamp and make certain very effective strategies. There are two schemes which use transaction time-stamps to

prevent deadlock. One is non-preemptive, called wait and die scheme, and one is preemptive called wound and die scheme.

Philosophically, very simple. Non-preemptive means that you be well behaved. So older transactions may wait for the younger ones to release the data item but the younger transaction never wait for the older ones. They are roll-backed immediately.

Preemptive is where an older transaction, instead of waiting for a younger transaction, kicks it out, wounds it out, but a younger transaction may wait for an older one. So, this is the philosophical, let me, let us look at it in informal terms.

(Refer Slide Time: 04:54)

Deadlock Prevention (2): Wait-Die Scheme

- It is a **non-preemptive** technique for deadlock prevention
- When transaction T_n requests a data item currently held by T_k , T_n is only if it has a timestamp smaller than that of T_k (That is, T_n is older than T_k , otherwise T_n is killed ("die"))
- If a transaction requests to lock a resource (data item), which is already locked by another transaction, then one of the two possibilities arises:
 - $\text{Timestamp}(T_n) < \text{Timestamp}(T_k)$: T_n , which is requesting a resource older than T_k , then T_n is allowed to "wait" until the data-item is released by T_k .
 - $\text{Timestamp}(T_n) > \text{Timestamp}(T_k)$: T_n is younger than T_k , then T_n is killed ("dies"). T_n is restarted later with a random delay but with the same timestamp.
- This scheme allows the older transaction to "wait" but kills the younger transaction.
- Example

So in non-preemptive, let us introduce the notation the transaction T_n is requesting for a data item, held by transaction T_k . So T_n is allowed to wait only if the time-stamp is smaller than

that of T_k . What does smaller time-stamp mean, that T_n started earlier so it is a older transaction, otherwise T_n is killed.

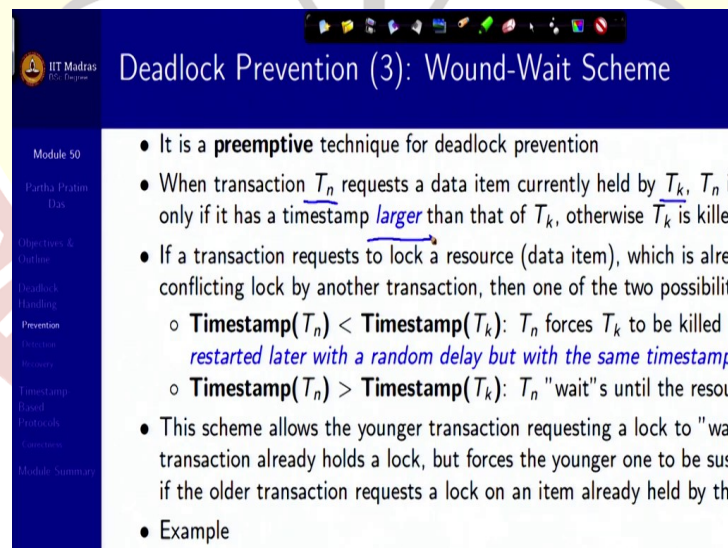
So what we can say in mathematical terms, if time-stamp T_n is less than times stamp T_k then T_n will wait while T_k is using the data item till it is available. But if it is more, that if it is newer then T_n is killed immediately. And what you do? You restart after a random delay.

But in that way what will happen? A young transaction, younger transaction will get even younger, even younger so to stop that when you restart you do not restart based on the current time when it is restarting you use the same time-stamp.

So I mentioned the time-stamp is indicative of the order, but is not exactly the clock time. So it is, it is actually starting at a later time but I am using the older time-stamp so that the seniority, the relative seniority of that transaction remains though the transaction has not been doing anything, so as if the transaction was sleeping during that time.

So it allows the older transaction to wait and kills the younger one. Here is an example which you can see for yourself. This is called the Wait-Die scheme, where things are non-preemptive.

(Refer Slide Time: 06:56)



Deadlock Prevention (3): Wound-Wait Scheme

- It is a **preemptive** technique for deadlock prevention
- When transaction T_n requests a data item currently held by T_k , T_n is only if it has a timestamp **larger** than that of T_k , otherwise T_k is killed
- If a transaction requests to lock a resource (data item), which is already conflicting lock by another transaction, then one of the two possibilities:
 - $\text{Timestamp}(T_n) < \text{Timestamp}(T_k)$: T_n forces T_k to be killed (restarted later with a random delay but with the same timestamp)
 - $\text{Timestamp}(T_n) > \text{Timestamp}(T_k)$: T_n "wait"s until the resource is available
- This scheme allows the younger transaction requesting a lock to "wait" until the older transaction already holds a lock, but forces the younger one to be suspended if the older transaction requests a lock on an item already held by the younger transaction
- Example



Deadlock Prevention (3): Wound-Wait Scheme

Module 50

Partha Pratim Das

Operations & Systems

Deadlocks

Handling

Prevention

Recovery

Timestamp

Deadlocks

Prevention

Recovery

Module Summary

- It is a **preemptive** technique for deadlock prevention
- When transaction T_n requests a data item currently held by T_k , T_n is only allowed to wait if it has a timestamp **larger** than that of T_k , otherwise T_k is killed
- If a transaction requests to lock a resource (data item), which is already held by another transaction, then one of the two possibilities:
 - **$\text{Timestamp}(T_n) < \text{Timestamp}(T_k)$** : T_n forces T_k to be killed (*restarted later with a random delay but with the same timestamp*)
 - **$\text{Timestamp}(T_n) > \text{Timestamp}(T_k)$** : T_n "wait"s until the resource is free
- This scheme allows the younger transaction requesting a lock to "wait" until the transaction already holds a lock, but forces the younger one to be suspended if the older transaction requests a lock on an item already held by the younger transaction
- Example

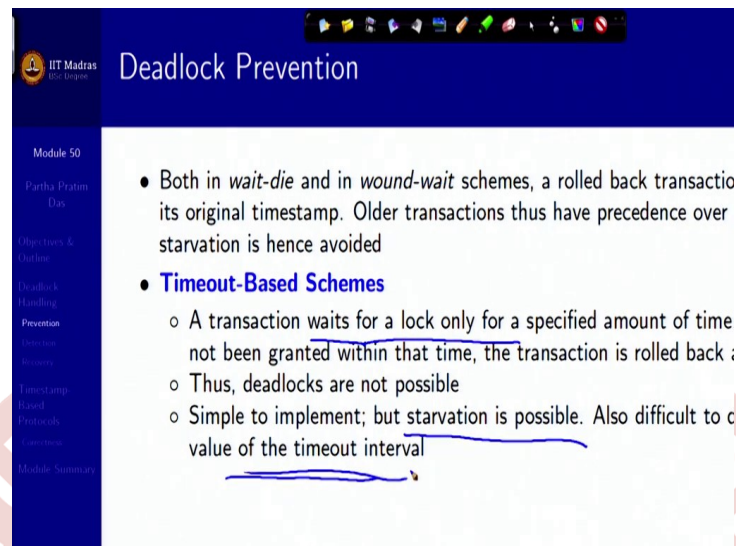
In the preemptive scheme, you do differently. The transaction T_n again requests a data item held by T_k . T_n is allowed to wait if it has a time-stamp larger than that of T_k . That is, if it is younger, otherwise T_k is killed the transaction which is, which was holding that data item, that itself is killed. That is why it is preemptive. In the earlier case we were killing the requesting transaction, in this case we are killing the transaction which is holding it.

So if you, if we look in mathematical terms, if this is the case that T_n actually is older, senior, so it is like you, you are sitting, you are in a room and there is one chair and you are sitting there. So if your, someone senior in your family would enter the room, you will leave the chair to that person, T_k is kicked out.

But if someone younger would come you will continue to be in that chair as long as you want, and only when you leave that younger person can take it. So that, that is the basic difference. So the T_n forces T_k to be killed, because T_n is senior in the time-stamp. And again T_k is started after a random delay with the same time-stamp, otherwise the T_n will wait until the resource is free, if it is.

So this is the preemptive scheme called wound and wait scheme. Again small example, you can see it for yourself. So these are very nice, simple schemes to ensure that deadlock will be prevented.

(Refer Slide Time: 09:00)



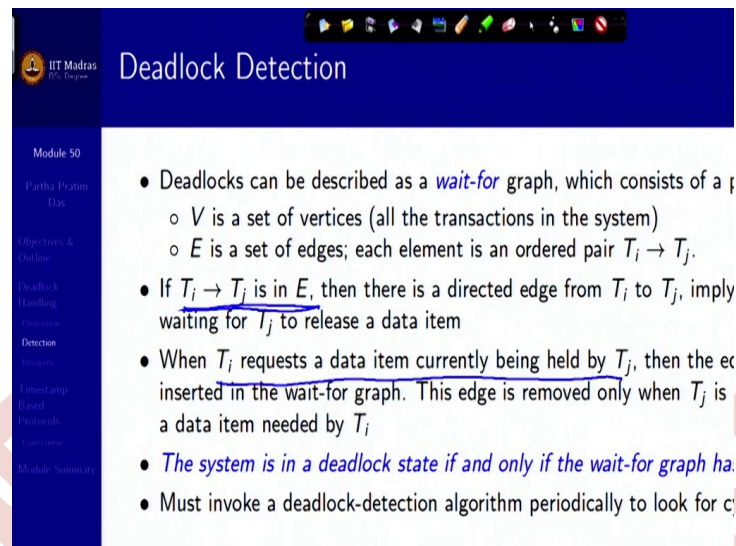
Deadlock Prevention

- Both in *wait-die* and in *wound-wait* schemes, a rolled back transaction has its original timestamp. Older transactions thus have precedence over starvation is hence avoided
- **Timeout-Based Schemes**
 - A transaction waits for a lock only for a specified amount of time; if not been granted within that time, the transaction is rolled back ;
 - Thus, deadlocks are not possible
 - Simple to implement; but starvation is possible. Also difficult to choose a good value of the timeout interval

Other time based schemes include like, a transaction might wait for a lock, for a specified amount of time. If it is not being granted then the transaction rolls back and restarts. Obviously, since the transaction by itself is, I mean, so this is kind of transaction committing suicide and coming back as a phoenix bird after a while, but certainly this will ensure that deadlock is not possible. But the starvation is still possible.

You may keep coming back and not find that resource free because since you are going out of the system you are not waiting, the time when you, when the lock is released and the data item becomes free may not be the time when you are around, and somebody else come and get it. So it is difficult to have a estimate for what is a good value of this time interval and so on. So for these reasons it is, it has been used but again not very.

(Refer Slide Time: 10:20)



Deadlock Detection

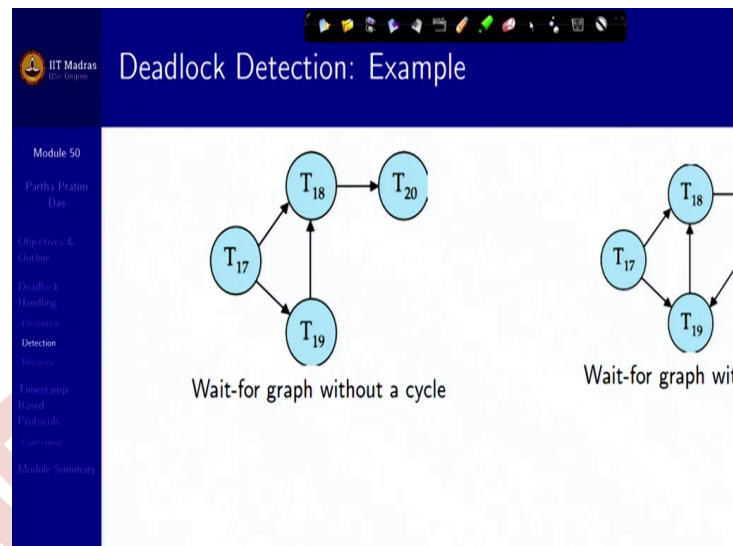
- Deadlocks can be described as a *wait-for* graph, which consists of a graph where:
 - V is a set of vertices (all the transactions in the system)
 - E is a set of edges; each element is an ordered pair $T_i \rightarrow T_j$.
- If $T_i \rightarrow T_j$ is in E , then there is a directed edge from T_i to T_j , implying T_i is waiting for T_j to release a data item
- When T_i requests a data item currently being held by T_j , then the request is inserted in the wait-for graph. This edge is removed only when T_j is a data item needed by T_i
- The system is in a deadlock state if and only if the wait-for graph has a cycle.*
- Must invoke a deadlock-detection algorithm periodically to look for cycles

The next question, this was about prevention, the next question is how you detect. Detection is, is actually very simple. It is very similar to the precedence graph idea. So if a transaction T_i waits on another T_j , then you draw an edge, so you create a graph where transactions are vertices and you have an edge for every waiting dependency of the transaction.

So if T_i requests a data item currently held by T_j , so this is the requesting edge. And the graph that results, and it is regularly updating. Certainly, it is a dynamic graph. So as the requests are coming, the, the locks are being granted, they are being released, always this graph is getting updated.

But you will know that the system is in deadlock if and only if this wait-for graph has a cycle. Otherwise, it cannot be because then it is an acyclic graph and the topological sorting has a way of actually allocating the locks as we have seen before.

(Refer Slide Time: 11:50)



So this wait-for graph is a good mechanism to dynamically keep track of, if a deadlock has happened because the system has to know that the deadlock has happened. Deadlock does not end. So unless the system can know immediately it will not be able to decide to tell the transactions to roll back. And then there are further questions in that.

(Refer Slide Time: 12:08)

Deadlock Recovery

- When deadlock is detected:
 - Some transaction will have to be rolled back (made a victim) to break that transaction as victim that will incur minimum cost
 - Rollback – determine how far to roll back transaction
 - Total rollback:** Abort the transaction and then restart it
 - More effective to roll back transaction only as far as necessary
 - Starvation happens if same transaction is always chosen as victim, so the number of rollbacks is included in the cost factor to avoid starvation

Deadlock Recovery

Module 50
Partha Pratim Das

- When deadlock is detected:
 - Some transaction will have to be rolled back (made a victim) to break that transaction as a victim that will incur minimum cost
 - Rollback – determine how far to roll back transaction
 - ▷ Total rollback: Abort the transaction and then restart it
 - ▷ More effective to roll back transaction only as far as necessary
 - Starvation happens if same transaction is always chosen as victim
 - number of rollbacks in the cost factor to avoid starvation

Handwritten diagram: $T_i \rightarrow A \rightarrow T_j$ (with a cross over T_j)

Deadlock Recovery

Module 50
Partha Pratim Das

- When deadlock is detected:
 - Some transaction will have to be rolled back (made a victim) to break that transaction as a victim that will incur minimum cost
 - Rollback – determine how far to roll back transaction
 - ▷ Total rollback: Abort the transaction and then restart it
 - ▷ More effective to roll back transaction only as far as necessary
 - Starvation happens if same transaction is always chosen as victim
 - number of rollbacks in the cost factor to avoid starvation

So now, when the deadlock is detected, then some transaction will have to be rolled back. There has to be a victim. Now, how do you select, because if I have a situation that this is T_i , this is T_j , this is A, this is B, that is, T_i has requested for A for which T_j is holding the lock, T_j has requested for B, for which T_i is holding the lock, then the only way to break this deadlock is either to kill this transaction, roll it back or to kill this transaction, roll things back.

As we do that, then the corresponding data item gets released and the other one can get it. So one way to say that, in principle what you want to say that, you should look for minimum cost. So one is also to determine how far to roll back. So it could be a total rollback, which is abort, or you could roll back the transaction only to as far as necessary, to break the deadlock.

How far is necessary, is to release the lock for which with the other transaction is waiting. And that is kind of the concept we discussed in terms of save point. This is basically, this is done in a similar manner by the lock points. So you have to roll back to that lock point which has caused this deadlock.

There may be several lock points in between which did not cause the deadlock, but once you have got the, that is the reason I was writing the data item on the arc, because you will also need to know on which data item this is happening.

So if I have this situation this T_j , and this is T_i , this is A, and I have this on B, and I, so T_i is waiting on T_j who is holding A. So if I decide to roll this back, then in T_j there will be some point where I have x-lock A or s-lock A, and then there may be another, several other locks on several other items, and this is the point where, where actually you have found the deadlock.

So at least you will have to roll back up to this point, because unless you release this lock that deadlock cannot, go away. So if you roll back less it, it is not enough, you still have the deadlock. But if you have gone to that lock point and released this, and have done enough to do this roll back in T_j it is sufficient. You do not need to always totally roll back, abort and roll back the entire transaction. So these are, these are, these are the trade-offs, different strategies that, that you can actually have.

The starvation would happen if some transaction is always chosen as victim. This is always possible. So what is done to avoid that is you include a cost factor in terms of, you are doing minimum cost, so include a cost factor which will keep track of how many roll backs you have done. So a transaction which is getting rolled back more number of times, the cost of throwing that transaction out becomes bigger, so eventually that transaction will come in and the starvation will end.

So all that, and you can see that here the, the basic strategy is like this wait-for graph or comparing time, and doing things are relatively elementary, common sense, and rest of it are basically a variety of heuristics which you, basically rule of thumb, which people have tried out and seen okay, okay this is.

Getting an absolute theory with the generalization of all transaction, all type of operation, all types of data, all types of concurrency has so far proved not to be very successful. But these strategies are very effective and that has made so much of distributed things.

(Refer Slide Time: 17:23)



Timestamp-Based Protocols

Module 50

Partha Pratim Das

Objectives & Outline

Deadlocks Handling

Timestamp-Based Protocols

Timestamp-Based Protocols

Timestamp-Based Protocols



Timestamp-Based Protocols

Module 50

Partha Pratim Das

Objectives & Outline

Deadlocks Handling

Timestamp-Based Protocols

Timestamp-Based Protocols

- Each transaction is issued a timestamp when it enters the system. If T_i has time-stamp $TS(T_i)$, a new transaction T_j is assigned time-stamp that $TS(T_i) < TS(T_j)$.
- The protocol manages concurrent execution such that the time-stamp serializability order
- In order to assure such behavior, the protocol maintains for each data values:
 - W-timestamp(Q)** is the largest time-stamp of any transaction that **write(Q)** successfully
 - R-timestamp(Q)** is the largest time-stamp of any transaction that **read(Q)** successfully



Timestamp-Based Protocols

Module 50

Partha Pratim Das

Objectives & Outline

Deadlocks Handling

Timestamp-Based Protocols

Timestamp-Based Protocols

- Each transaction is issued a timestamp when it enters the system. If T_i has time-stamp $TS(T_i)$, a new transaction T_j is assigned time-stamp that $TS(T_i) < TS(T_j)$.
- The protocol manages concurrent execution such that the time-stamp serializability order
- In order to assure such behavior, the protocol maintains for each data values:
 - W-timestamp(Q)** is the largest time-stamp of any transaction that **write(Q)** successfully
 - R-timestamp(Q)** is the largest time-stamp of any transaction that **read(Q)** successfully

Before I close on this, let me also present a, a little bit of a different way of managing the read-write, which can be useful. So what we say is, again each transaction has a time-stamp, as it enters the system. An old transaction T_i has a time-stamp $TS(T_i)$, another has $TS(T_j)$. So you have this, $TS(T_i)$ is less than $TS(T_j)$, so $TS(T_i)$ is an older transaction.

(Refer Slide Time: 18:08)

Timestamp-Based Protocols

- Each transaction is issued a timestamp when it enters the system. If T_i has time-stamp $TS(T_i)$, a new transaction T_j is assigned time-stamp that $TS(T_i) < TS(T_j)$.
- The protocol manages concurrent execution such that the time-stamp serializability order
- In order to assure such behavior, the protocol maintains for each data values:
 - **W-timestamp(Q)** is the largest time-stamp of any transaction that **write(Q)** successfully
 - **R-timestamp(Q)** is the largest time-stamp of any transaction that **read(Q)** successfully

Now here, this protocol says that you will manage the concurrent execution based on the time-stamps. Earlier we were using time-stamp to, like in wait-die, or wound-die we use time-stamp to decide who should wait, who should not wait, who should be killed and so on, or we used some of that to commit suicide by a, but here we are saying that the time-stamp will decide the concurrency.

So for that it maintains two different time-stamp with every data item Q . This is called write timestamp and read timestamp. This is the latest, last time that this data item was written, is a write timestamp. So this is the largest time-stamp which means the most recent write of that data that was successful.

Similarly, the most recent, largest time-stamp of any transaction, we do not care which transaction, but any transaction what is the latest time that Q was written W-timestamp Q , what is the latest time R , Q was read, the largest time, latest time for read.

(Refer Slide Time: 19:51)

Timestamp-Based Protocols (2)

- The timestamp ordering protocol ensures that any conflicting **read** operations are executed in timestamp order
- Suppose a transaction T_i issues a **read**(Q)
 - a) If $TS(T_i) \leq W\text{-timestamp}(Q)$, then T_i needs to **read** a value of Q that has not been overwritten
 - Hence, the **read** operation is rejected, and T_i is rolled back.
 - b) If $TS(T_i) \geq W\text{-timestamp}(Q)$, then the **read** operation is executed. $R\text{-timestamp}(Q)$ is set to $\max(R\text{-timestamp}(Q), TS(T_i))$.

Timestamp-Based Protocols (2)

- The timestamp ordering protocol ensures that any conflicting **read** operations are executed in timestamp order
- Suppose a transaction T_i issues a **read**(Q)
 - a) If $TS(T_i) \leq W\text{-timestamp}(Q)$, then T_i needs to **read** a value of Q that has not been overwritten
 - Hence, the **read** operation is rejected, and T_i is rolled back.
 - b) If $TS(T_i) \geq W\text{-timestamp}(Q)$, then the **read** operation is executed. $R\text{-timestamp}(Q)$ is set to $\max(R\text{-timestamp}(Q), TS(T_i))$.

Timestamp-Based Protocols (2)

- The timestamp ordering protocol ensures that any conflicting **read** operations are executed in timestamp order
- Suppose a transaction T_i issues a **read**(Q)
 - a) If $TS(T_i) \leq W\text{-timestamp}(Q)$, then T_i needs to **read** a value of Q that has not been overwritten
 - Hence, the **read** operation is rejected, and T_i is rolled back.
 - b) If $TS(T_i) \geq W\text{-timestamp}(Q)$, then the **read** operation is executed. $R\text{-timestamp}(Q)$ is set to $\max(R\text{-timestamp}(Q), TS(T_i))$.

And then, along with the timestamp of the transaction, you decide on when to proceed, when to roll back. Suppose, now, so I have the, I have the transaction T_i , and it has a time-stamp $TS(T_i)$. I want to do a read. Now, when I want to do a read, what do I need? I need to use a data which is correct, updated recent.

So there are two conditions possible. This may be less than the write timestamp or this may be greater than the write timestamp. If it is less, what does it mean? That this transaction was started before this write happened. So T_i needs to read a value of Q that is already overwritten because that, what is W-timestamp, is it the last write, the time it was last written. So your transaction is older than that. So it does not make sense to do this read. So the read is rejected, T_i is rolled back.

On the other hand, if it is more than W-timestamp, then the read operation is meaningful, that is your transaction started after this write happened, so that is the latest value, and therefore now you are using a fresh value, not one which may have been invalidated later.

So the read is allowed, and since you do a read the read timestamp of Q has to change. Now what, what will it change to? The current time, this is the transaction time that this transaction has. This is the time of the transaction after that write.

So it could be that, or it is possible that it has a read timestamp which is higher, which means that it was a write, then you had T_i , then you had read. So it will, when it does that it will take the maximum of these two. This is your protocol to read.

(Refer Slide Time: 23:20)

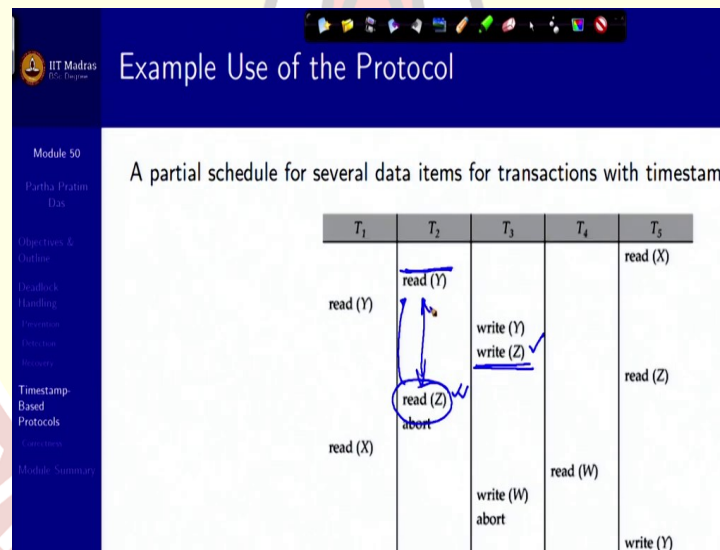
Timestamp-Based Protocols (3)

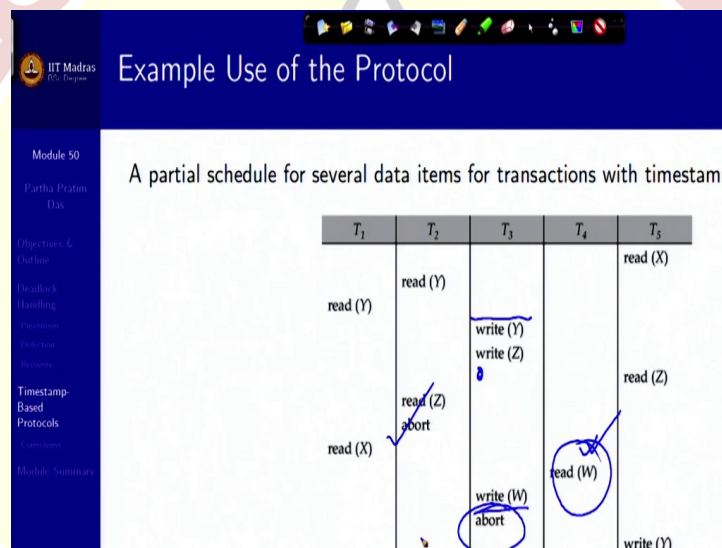
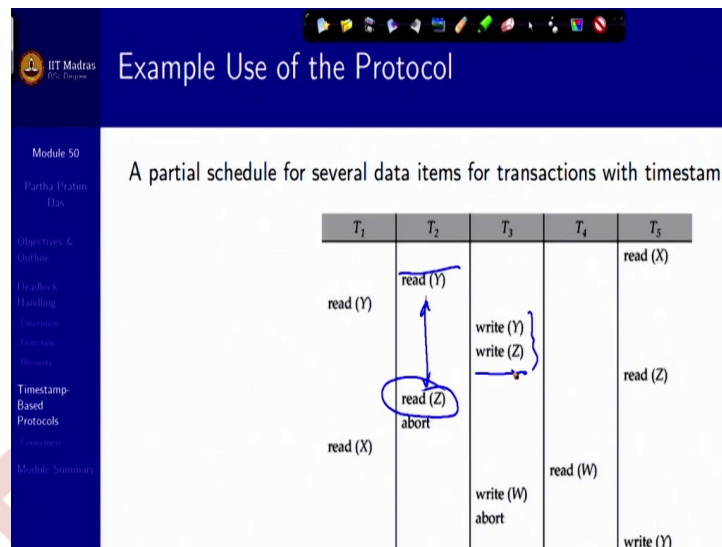
- Suppose that transaction T_i issues $write(Q)$.
 - If $TS(T_i) < R\text{-timestamp}(Q)$, then the value of Q that T_i is proc previously, and the system assumed that that value would never b
 - ▷ Hence, the **write** operation is rejected, and T_i is rolled back
 - If $TS(T_i) < W\text{-timestamp}(Q)$, then T_i is attempting to write an Q
 - ▷ Hence, this **write** operation is rejected, and T_i is rolled back
 - Otherwise, the **write** operation is executed, and $W\text{-timestamp}(Q)$

What do I have to do for write? You want to write this. Now, if $TS(T_i)$, the time-stamp of T_i is less than the latest read time, which means this is an older transaction, so you are trying to change a value which has already been read, which was needed previously. So the write has to be rejected. If $TS(T_i)$ is less than the W-timestamp, so this is older transaction, after that, after that transaction Q has been written, that is the reason it is more. So it is trying to write an obsolete value.

So this also should be rejected. So what means that if $TS(T_i)$ is greater than read timestamp, as well as write timestamp, it is more recent than both of these operations, then only you will be allowing a write operation. And naturally as you do the write, the write time-stamp would be set to the $TS(T_i)$. Very simple time based protocol, easy to implement, easy to understand, and but it guarantees that you will not have deadlock, and you are having the serializability according to this.

(Refer Slide Time: 25:10)





For example, here your, these are, these are different transactions and from the first statements, you will know what is the start time. For example, this read(Z), this read(Z), so at this point what will be the read time, write time for Z is this one. And what is the time for this transaction, is T2, here. So T2 is an older transaction and after that this has happened, write has happened.

So this read was intended to be, is intended to be, the fact that it is happening now is because you are having the concurrency. But it is intended to be here. You are getting the point? It is not that it is, it is here, not that because this gap.

This gap is not desired by the transaction. In the transaction is read(Y), read(Z). Like you have write(Y), write(Z), but because of the concurrency, this transaction did not get to work. It did not get the processor, CPU. Other transactions worked.

So when it is getting this, its intended time is attributed at the beginning of the transaction, which is old. So it needed to read the data then but that data has changed in this write(Z). So this read is not valid. So you abort. That is the kind of logic.

Similar thing is happening here, in this write. So this is your transaction start state. So it should have happened here. If it would have happened here, then T4 would have read something different. But T4 has already read this.

So now if you change, then you are getting inconsistent with T4. So you cannot allow this write to happen, you abort. You check the other cases, they satisfy, and therefore those will not be aborted.

(Refer Slide Time: 28:07)

Correctness of Timestamp-Ordering Protocol

- The timestamp-ordering protocol guarantees serializability since all the precedence graph are of the form:

Diagram illustrating a precedence graph:

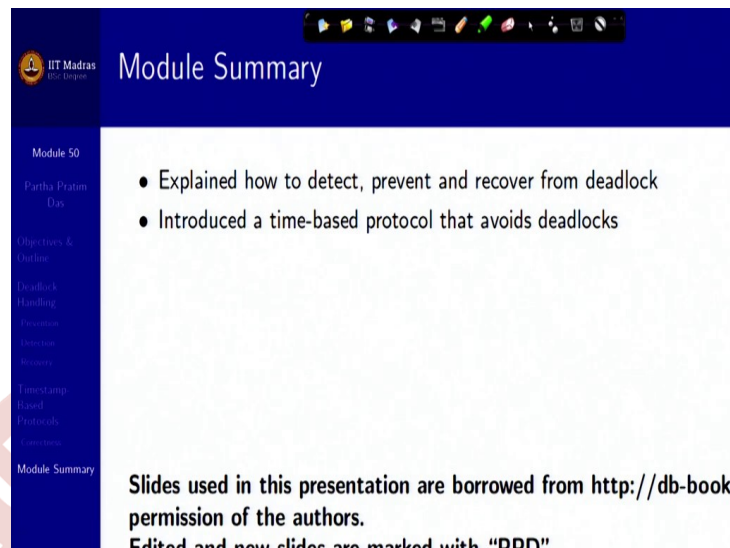
```
graph LR; A((transaction with smaller timestamp)) --> B((transaction with larger timestamp))
```

Thus, there will be no cycles in the precedence graph

- Timestamp protocol ensures freedom from deadlock as no transaction
- But the schedule may not be cascade-free, and may not even be recoverable

So this time based protocol guarantees serializability, because all, all are in terms of transaction with smaller time-stamp is dependent for something which for transaction with larger time-stamp. So there cannot be any, any cycle in this precedence graph. And that ensures freedom from deadlock as no transaction ever waits. But this schedule may not be cascade free. In fact, there are given instances where it is not even recoverable. So those are some of the other perils that you come in here.

(Refer Slide Time: 28:48)



Module Summary

- Explained how to detect, prevent and recover from deadlock
- Introduced a time-based protocol that avoids deadlocks

Slides used in this presentation are borrowed from <http://db-book> permission of the authors.
Edited and new slides are marked with "DDN"

With that we come to the end of this module, and we have discussed about detection, prevention and recovery, partly, just glimpses, from deadlock and a time based protocol that guarantees a deadlock free processing. So these are, these are some of the different strategies which enhance the concurrency while keeping deadlock under control, for better throughput of the transaction processing system.

So with that we will close this week's discussion on transactions and concurrency, and get into a new topic in the next module, next week. Thank you very much for your attention. See you in the next module.